Introduction à l'IA / Intelligent Systems

Symbolic AI – Constraint Programming

Sylvain Bouveret (sylvain.bouveret@grenoble-inp.fr)
April 2024

Introduction

This document contains several practical exercises on constraint programming. We will use *Google OR-Tools*[1] for this session. All sections are independent on each other and of equal complexity. You can do the exercises in any order. However:

- I strongly advise you to start with Section 1 unless you already perfectly know constraint programming.
- I also advise you to do the N-queens (Section 2) problem just after, because it will give you some insight about how to use global constraints.

This practical work will not be graded.

1 Discovering OR-Tools

1.1 Installing the environment

The *OR-Tools* library is not installed by default, but you can simply install it using *pip*. We advise you to use a Python virtual environment to keep the installation local, but it will depend on whether `venv` is installed on your machine (it might not be on Ensimag machines).

If `venv` is already installed:

```
$ python3 -m venv venv
[...]
$ . venv/bin/activate
[...]
$ pip3 install ortools
[...]
$ # Run your programs here, and when you are done:
$ deactivate
```

Otherwise (if the first line above fails), just install *OR-Tools* user-wise:

```
$ pip3 install ortools
```

1.2 Learning OR-Tools

OR-Tools aims at embedding constraint programming in several languages (like Python). It can be used like most CP systems:

- first, you declare the variables of your problems;
- then, you declare the constraints;
- finally, you run the solver that will return one solution (or all of them) if it exists.

[1] https://developers.google.com/optimization

As we have seen during the lecture, constraint programming is a declarative paradigm, just like logic programming. In other words, you tell the solver what you want, but never how to find a solution.

Question 1 After this short introduction, it is time for you to experiment. To learn how *OR-Tools* works, follow the guide at:

`https://developers.google.com/optimization/cp`.

2 The N-queens problem

In this section, we will develop a model for the N-queens problem. As a reminder, the problem is about placing N queens on a N by N chessboard so that no two of them attack each other. See Figure 1 for an example of a solution for the 8-queens problem.

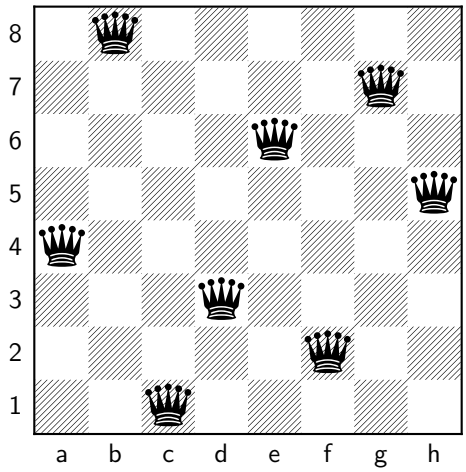

Figure 1: A solution for the 8-queens problem.

Question 2 Write a first model of the N-queens problem only based on binary constraints (of kind $x \neq y$). Run the solver to find a solution.

Question 3 Extend your program so that:

1. the board size (N) can be set when running the program, by using a command line argument
2. when the solver find a solution, this solution is pretty printed in the terminal (for instance as a grid representing the chessboard).

Question 4 No write a second model of the N-queens problem based on global constraints.

Question 5 Compare the solving time of both models for various values of N. Do you see any difference between both approaches? Which one is the fastest?

3 A Sudoku solver

In this section, we will do a solver for the Sudoku problem. If you do no know what this game is about, just do a quick search on the Web.

In this section, we will use the library *Dokusan*[2] to randomly generate Sudoku grids.[3]

First install the library using `pip`:

```
$ pip3 install dokusan
```

Once the library is installed, a Sudoku grid can easily be generated as follows:

```
from dokusan import generators

board = generators.random_sudoku(avg_rank=150)
# Here, avg_rank is the grid difficulty
```

After having generated a grid and stored it in a variable called `board`, you can access the value of any cell (i, j) $(i \geq 0, j \geq 0)$ with `board[(i, j)].value`. This value is `None` if the value is not set in this cell, and is the integer value otherwise.

Question 6 Write a Sudoku solver using *OR-Tools*.

4 Cryptarithmetic

A cryptarithmetic puzzle is an arithmetic operation where the digits of all the numbers are replaced by letters. The aim is to find an assignment of letters to digits so that the operation is mathematically valid. Let us give an example:

```
   SEND          9567      (S → 9, E → 5, N → 6, D → 7,
+  MORE    →   + 1085       M → 1, O → 0, R → 8, Y → 2)
=======        =======
  MONEY         10652
```

For the solution to such a puzzle to be valid:

1. all the letters have to be mapped to different digits;
2. the letters corresponding to the leading digits in all the numbers cannot be zero.

To simplify the problem, we will assume that:

- it is in base 10;
- the operator can only be "+";
- however, there can be any number of operands.

Question 7 Write a generic solver for any cryptarithmetic puzzle. The solver should be able to solver any puzzle given as input in the following form: `"SEND + MORE = MONEY"`.

[2] https://pypi.org/project/dokusan/

[3] Note that this library also has a simple solver, although we will not use it today.

5 Winter is coming (again)

Remember *Game of Thrones*? We worked on it during the last practical session. Here, we will go back to this example, to show you how constraint programming can be used to do some similar reasoning tasks as we did in datalog. Although we cannot express general recursive rules in constraint programming, we will see that CP is still powerful enough to encode interesting queries.

Today, we will still use the same dataset of characters with their family relations and houses we used before. You can still download it on Caseine (the file is called `got.csv`).[4]

In this exercise, the characters will be encoded as integers, since *OR-Tools* only deals with integer variables.

Question 8 Write a Python program to read the CSV file and load the data on it. In the end, each character encountered should have a unique number (between 0 and the number of characters - 1), and we should have stored as lists of pairs of integers all the relations between child and mother, and between child and father.

Now we will create a CP model to encode queries about the characters and their family relations.

Question 9 Create two CP variables `x` and `y` and add a constraint to encode that `y` is `x`'s parent.[5]

Use a solver to print the list of all pairs (child, parent) of the dataset.

Adapt your solver to only print Arya Stark's parents.

Question 10 Adapt your program to find Arya Stark's grandparents, great-grandparents, great-great-grandparents, and so on (until some fixed generation of ancestors).

Question 11 Write another model to find every pair of characters that are both siblings and cousins?

[4] As s reminder, this file has been adapted by myself from a file provided by Shirin Elsinghorst (`https://shiring.github.io/networks/2017/05/15/got_final`), itself adapted from an initial Kaggle dataset (`https://www.kaggle.com/mylesoneill/game-of-thrones`).

[5] Note that you can use method `add_allowed_assignments(...)` to define a constraint by explicitly listing all the allowed assignments.